

Stop Fighting With OCR: Convert Any Document to Markdown, HTML, or JSON Using SmolDocling



Bibek Poudel · [Follow](#)

6 min read · 2 days ago



Listen



Share



More



Parse any
documents
with
SmolDocling

If you've spent hours testing OCR tools like Tesseract or Textract, only to see them **mangle tables, skip headers, or flatten document structure**, you're not alone.

OCR still works great for plain text. But once your documents include multi-column layouts, code blocks, equations, or even structured lists — traditional OCR tools fall

short.

What we need is something smarter. Something that doesn't just read text, but understands the **layout, semantics, and structure** of a document.

And that's where **SmolDocling** and the **Docling library** come in.

What Are SmolDocling and Docling?

- **SmolDocling** is a 256M-parameter Visual Language Model that reads document pages as images and outputs **semantic markup** called **DocTags**.
- **Docling** is a Python toolkit that takes these DocTags and converts them to structured formats.

SmolDocling replaces traditional OCR with a smarter, layout-aware approach that **understands** documents.

What Are DocTags?

Here's a snippet of what SmolDocling outputs after processing a document page:

```
<heading level="1">Introduction</heading>
<paragraph>SmolDocling is a compact vision-language model for document understa
<table>
  <row><cell>Model</cell><cell>Params</cell></row>
  <row><cell>SmolDocling</cell><cell>256M</cell></row>
</table>
```

DocTags aren't just about layout — they **preserve semantics, reading order, and hierarchy**. Which means downstream tools like Docling can do more accurate conversions than any post-hoc heuristic-based parser.

Setup Instructions (Mac M1/M2/M3/M4 Recommended)

This tutorial is tested on a MacBook M4 and uses [MLX](#), Apple's Metal-accelerated ML backend.

It will still work on other platforms, but you may need to modify a few model-loading lines to use PyTorch or Hugging Face directly.

Step 1: Create Python Environment

```
# Install uv if you haven't yet
pip install uv

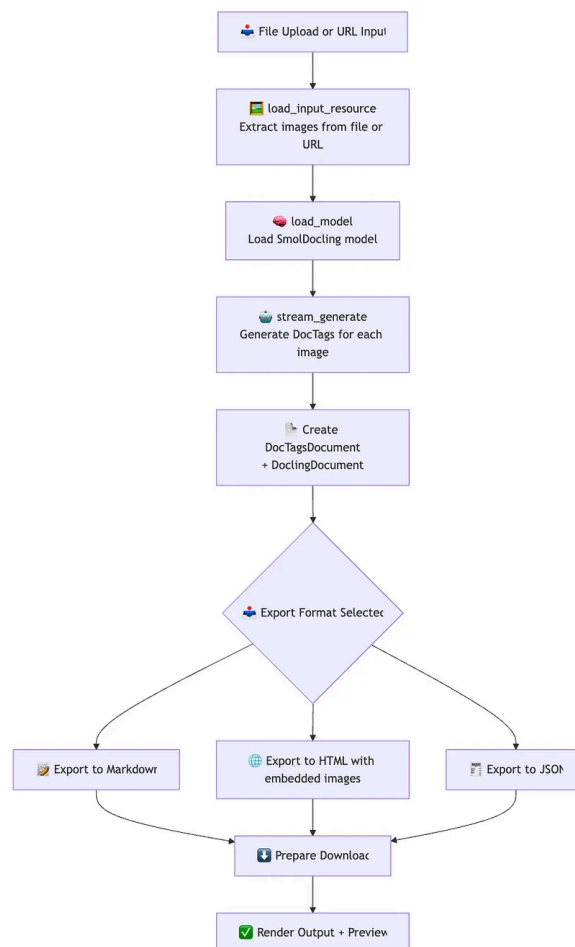
# Create a virtual environment
uv venv smoldocling-env
source smoldocling-env/bin/activate
```

Step 2: Install Required Packages

```
uv pip install gradio mlx-vlm docling-core pillow pdf2image requests
brew install poppler
```

Building the App

This is going to be our setup:



1. Load PDFs or Images (local or via URL)

```
from PIL import Image
from pathlib import Path
from urllib.parse import urlparse
from pdf2image import convert_from_path, convert_from_bytes
import requests
from io import BytesIO

def load_input_resource(input_path):
    images = []
    if urlparse(input_path).scheme != "":
        response = requests.get(input_path, stream=True, timeout=10)
        content = BytesIO(response.content)
        if content.read(4) == b"%PDF":
            content.seek(0)
            images.extend(convert_from_bytes(content.read()))
        else:
            content.seek(0)
            images.append(Image.open(content))
    else:
        path = Path(input_path)
        if path.suffix.lower() == ".pdf":
            images.extend(convert_from_path(str(path)))
        else:
            images.append(Image.open(path))
    return images
```

2. Load the SmolDocling Model (with MLX backend)

```
import mlx.core as mx
from mlx_vlm import load
from mlx_vlm.utils import load_config

def load_model():
    mx.set_default_device(mx.gpu)
    model_path = "ds4sd/SmolDocling-256M-preview-mlx-bf16"
    model, processor = load(model_path)
    model.eval()
    mx.eval(model.parameters())
    config = load_config(model_path)
    return model, processor, config
```

3. Process Documents

```

def process_document(file_obj, url_input, export_format):
    """Process a document with SmolDocling and return the results."""
    try:
        # Load the model
        model, processor, config = load_model()

        # Determine the input source
        if file_obj is not None:
            # Save the uploaded file to a temporary location
            temp_dir = tempfile.mkdtemp()

            # Get the file name from the upload
            file_name = getattr(file_obj, 'name', 'uploaded_file')

            # Handle different types of file objects that gradio might provide
            temp_path = os.path.join(temp_dir, file_name)

            # Different handling based on file object type
            if hasattr(file_obj, 'read'):
                # If it's a file-like object with read method
                with open(temp_path, "wb") as f:
                    f.write(file_obj.read())
            else:
                # If it's already a path
                if isinstance(file_obj, str):
                    temp_path = file_obj
                else:
                    # For Gradio's file component that returns tuple (path, name)
                    temp_path = file_obj[0] if isinstance(file_obj, tuple) else file_obj

            input_path = temp_path
        elif url_input.strip():
            input_path = url_input.strip()
        else:
            return "Please provide either a file upload or a URL", None, None

        # Get images from input file
        images = load_input_resource(input_path)
        if not images:
            return "No images could be extracted from the provided file or URL"

        # Set up the prompt
        prompt = "Convert this page to docling."
        formatted_prompt = apply_chat_template(processor, config, prompt, num_images=1)

        # Process each image and generate output
        all_outputs = []
        all_images = []
        processing_log = ""

```

```

for i, image in enumerate(images):
    processing_log += f"Processing page {i+1}/{len(images)}...\n\n"
    processing_log += "DocTags:\n\n"

    output = ""
    all_images.append(image)

    for token in stream_generate(
        model, processor, formatted_prompt, [image], max_tokens=4096, v
    ):
        output += token.text
        if "</doctag>" in token.text:
            break

    all_outputs.append(output)
    processing_log += output + "\n\n"

# Create DoclingDocument
doctags_doc = DocTagsDocument.from_doctags_and_image_pairs(all_outputs,
doc = DoclingDocument(name="ProcessedDocument")
doc.load_from_doctags(doctags_doc)

# Export based on selected format
if export_format == "Markdown":
    result = doc.export_to_markdown()
elif export_format == "HTML":
    html_output = tempfile.NamedTemporaryFile(suffix=".html", delete=False)
    html_path = Path(html_output.name)
    doc.save_as_html(html_path, image_mode=ImageRefMode.EMBEDDED)
    with open(html_path, "r") as f:
        result = f.read()
elif export_format == "JSON":
    doc_dict = doc.export_to_dict()
    result = json.dumps(doc_dict, indent=4)
else:
    result = "Invalid export format selected"

# Return the first image as a preview and the processing log
return result, images[0] if images else None, processing_log

except Exception as e:
    import traceback
    error_details = traceback.format_exc()
    return f"Error processing document: {str(e)}\n\nDetails:\n{error_detail

```

4. Render output

```

def render_output(result, export_format):
    """Render the processed result based on export format."""
    if export_format == "Markdown":
        # For markdown, show the rendered markdown component.
        return gr.update(value=result, visible=True), gr.update(visible=False),
    elif export_format == "HTML":
        # For HTML, render as an embedded web component.
        return gr.update(visible=False), gr.update(value=result, visible=True),
    elif export_format == "JSON":
        # For JSON, parse it into an object so that gr.JSON can render it as an
        try:
            json_obj = json.loads(result)
        except Exception as e:
            json_obj = {"error": "Invalid JSON", "detail": str(e)}
        return gr.update(visible=False), gr.update(visible=False), gr.update(va
    else:
        # Fallback: hide all rendered views.
        return gr.update(visible=False), gr.update(visible=False), gr.update(vi

def prepare_download(result, export_format):
    """Prepare a downloadable file for the processed output."""
    if export_format == "Markdown":
        ext = ".md"
    elif export_format == "HTML":
        ext = ".html"
    elif export_format == "JSON":
        ext = ".json"
    else:
        ext = ".txt"
    # Create a temporary file with the correct file type.
    temp_file = tempfile.NamedTemporaryFile(delete=False, suffix=ext)
    temp_file.write(result.encode("utf-8"))
    temp_file.close()
    # Return update objects for the download buttons.
    return gr.update(value=temp_file.name), gr.update(value=temp_file.name)

```

5. Gradio UI for One-Click Use

```

# Create the Gradio interface
with gr.Blocks(title="SmolDocling Document Processing") as app:
    # Add custom CSS for border styling in the output sections.
    gr.HTML(
        """
        <style>
        #raw_output_box, #formatted_output_box {
            border: 1px solid #ccc;
            padding: 10px;

```

```

        border-radius: 5px;
    }
</style>
"""
)

lang=None
with gr.Row():
    with gr.Column(scale=1):
        file_input = gr.File(label="Upload PDF or Image")
        url_input = gr.Textbox(label="Or enter a URL to a PDF or Image")
        export_format = gr.Radio(
            choices=["Markdown", "HTML", "JSON"],
            label="Export Format",
            value="Markdown"
        )
        submit_button = gr.Button("Process Document", variant="primary")
    if export_format == "Markdown":
        lang = "markdown"
    elif export_format == "HTML":
        lang = "html"
    elif export_format == "JSON":
        lang = "json"
    with gr.Column(scale=2):
        with gr.Tab("Raw Output"):
            with gr.Column(elem_id="raw_output_box"):
                # Display the raw output in a code block.
                output_text = gr.Code(label="Structured Output", language=lang)
                download_raw = gr.DownloadButton("Download Raw Output")
        with gr.Tab("Document Preview"):
            preview_image = gr.Image(label="Document Preview", type="pil")
        with gr.Tab("Log"):
            # Display the log in a code block.
            log_output = gr.Code(label="Processing Log", language="html", 1)
        with gr.Tab("Formatted Output"):
            with gr.Column(elem_id="formatted_output_box"):
                rendered_markdown = gr.Markdown(visible=False, label="Markdown Render")
                rendered_html = gr.HTML(visible=False, label="HTML Render")
                rendered_json = gr.JSON(visible=False, label="JSON Render")
                download_formatted = gr.DownloadButton("Download Formatted")

# Set up event handlers with chained callbacks:
submit_button.click(
    process_document,
    inputs=[file_input, url_input, export_format],
    outputs=[output_text, preview_image, log_output]
).then(
    render_output,
    inputs=[output_text, export_format],
    outputs=[rendered_markdown, rendered_html, rendered_json]
).then(

```



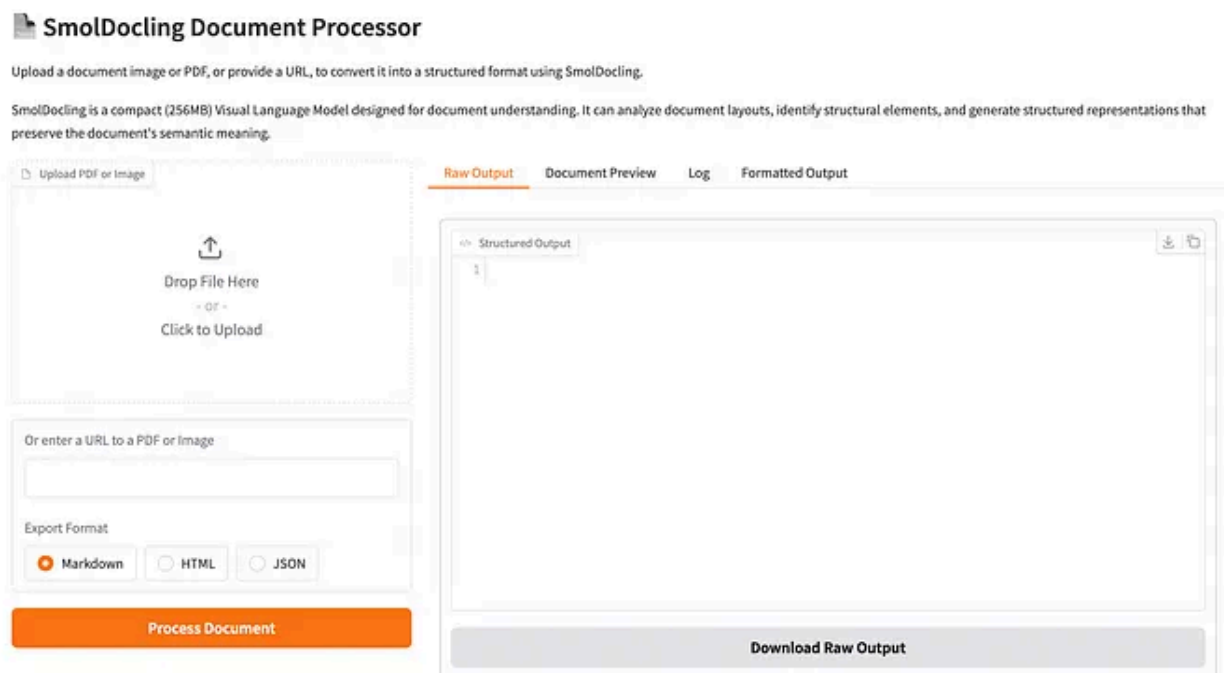
```
        prepare_download,  
        inputs=[output_text, export_format],  
        outputs=[download_raw, download_formatted]  
    )  
  
if __name__ == "__main__":  
    app.launch()
```

Running the App

1. Run the app with following command.

```
uv run main.py
```

The app will run at <http://127.0.0.1:7860/>



2. Use any of the example images here for testing:

https://github.com/docling-project/docling-ibm-models/blob/main/tests/test_data/samples/page_with_table.png

https://github.com/docling-project/docling-ibm-models/blob/main/tests/test_data/samples/page_with_list.png

SmolDocling Document Processor

Upload a document image or PDF, or provide a URL, to convert it into a structured format using SmolDocling.

SmolDocling is a compact (256MB) Visual Language Model designed for document understanding. It can analyze document layouts, identify structural elements, and generate structured representations that preserve the document's semantic meaning.

Upload PDF or image

page_with_table.png 450.3 KB

Or enter a URL to a PDF or image

Export Format

☒ Markdown ☐ HTML ☐ JSON

Process Document

Raw Output Document Preview Log Formatted Output

Structured Output

```

1 order to compute the TED score. Inference timing results for all experiments were obtained from the same machine on a si
2
3 ## 5.1 Hyper Parameter Optimization
4
5 We have chosen the PubTabNet data set to perform HPO, since it includes a highly diverse set of tables. Also we report F
6
7 Table 1. HPO performed in OTSL and HTML representation on the same transformer-based TableFormer [9] architecture, train
8
9 | # | # | # | TEDs | TEDs | mAP | Inference time (secs) |
10 |-----|-----|-----|-----|-----|-----|-----|
11 | enc-layers | dec-layers | Language | simple | complex | all | (0.75) | time (secs) |
12 | 6 | 6 | OTSL | 0.965 | 0.934 | 0.955 | 0.88 | 2.73 |
13 |  |  | HTML | 0.969 | 0.927 | 0.955 | 0.857 | 5.39 |
14 | 4 | 4 | OTSL | 0.938 | 0.904 | 0.927 | 0.853 | 1.97 |
15 |  |  | HTML | 0.952 | 0.909 | 0.938 | 0.843 | 3.77 |
16 | 2 | 4 | OTSL | 0.923 | 0.897 | 0.915 | 0.859 | 1.91 |
17 |  |  | HTML | 0.945 | 0.901 | 0.931 | 0.834 | 3.01 |
18 | 4 | 2 | OTSL | 0.952 | 0.92 | 0.942 | 0.857 | 1.22 |
19 |  |  | HTML | 0.984 | 0.903 | 0.931 | 0.824 | 2 |
20 |  |  |  |  |  |  |  |  |

```

Download Raw Output

You can preview your converted code in the *Formatted Output* Tab.

Also, if you want to check the doc tags that was generated, it will be in the *Log* Tab.

Complete Code

You can find the complete code at my Github repository:

<https://github.com/bibekplus/SmolDocling-Docment-Processor>

[Open in app ↗](#)

Medium

Search



models that run beautifully on Apple silicon.

If your workflow involves:

- Academic papers
- Business reports
- Form digitization
- Document-based LLM agents

Give SmolDocling a try. :-)

Resources

- [SmolDocling Paper \(arXiv\)](#)
- [Docling GitHub](#)

- [MLX GitHub](#)
- [Gradio](#)

Ocr

Docling

Smoldocling

Large Language Models

Llm



Follow

Written by Bibek Poudel

88 Followers · 8 Following

AI/ML Engineer

No responses yet



Sandesh Chand

What are your thoughts?

More from Bibek Poudel



 Bibek Poudel

Create your own chatbot with Llama2, Ollama and Gradio

Overview

Dec 1, 2023  116  3



 Bibek Poudel

Running Gemma3 Locally with llama.cpp

In this guide, I will walk you through running the Gemma3 model locally using llama.cpp installed via Homebrew. You'll learn how to run...

Mar 25

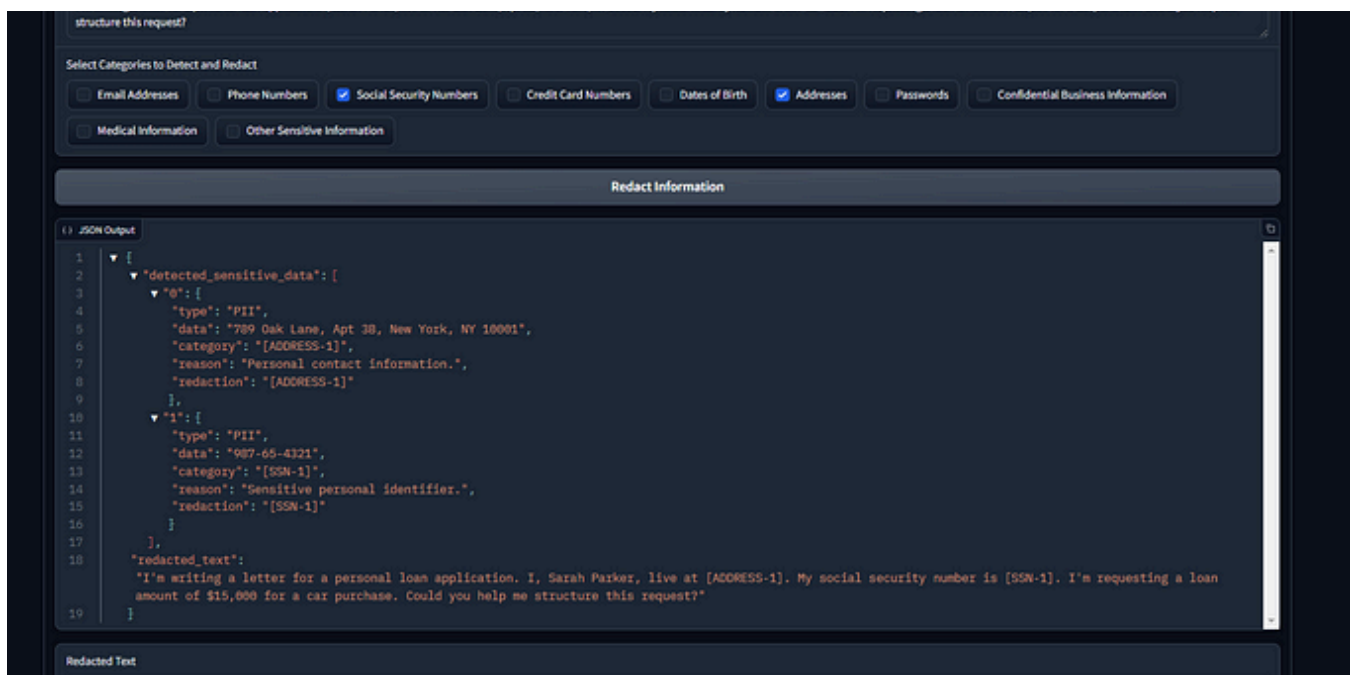


Bibek Poudel

What is MCP and How to Create Your Own MCP Server: A Simple Guide

In this guide, I'll try to explain what the Model Context Protocol (MCP) is in simple language and walk you through creating your own MCP...

6d ago



Bibek Poudel

Creating a Secure Local Guardrail System Using Ollama, LangChain, and Llama 3.1–8b

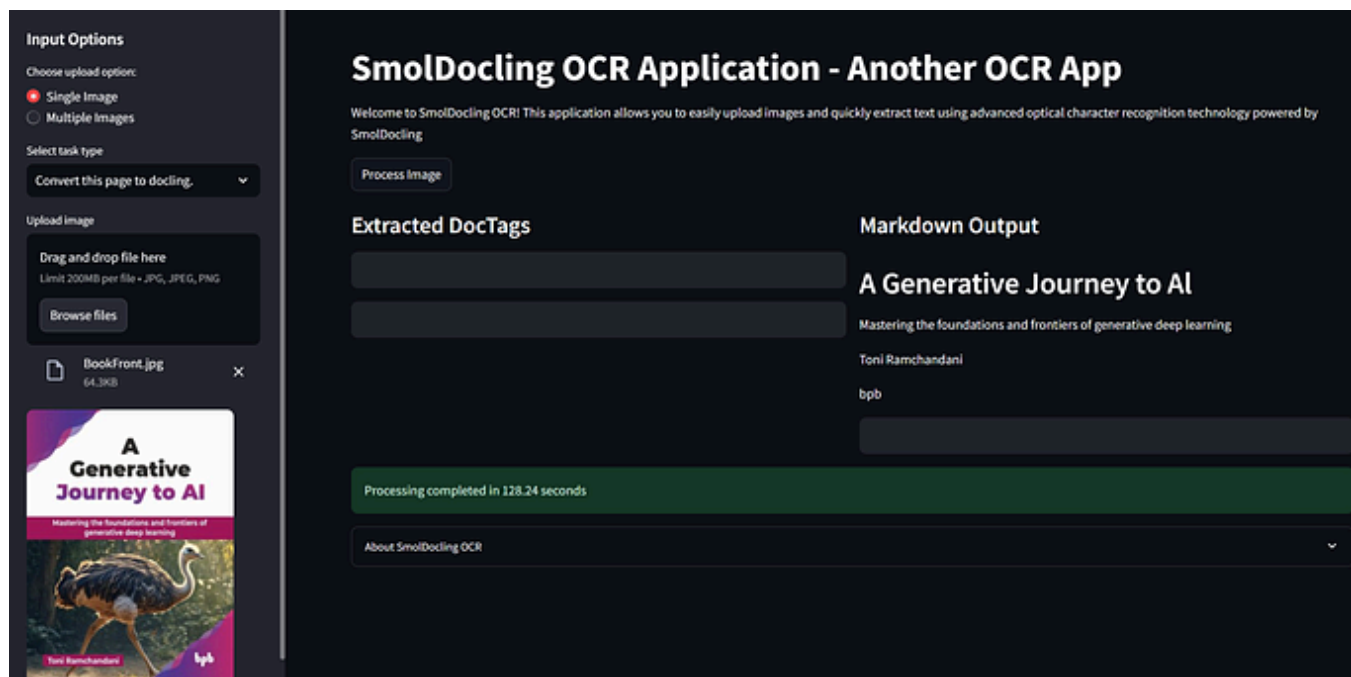
In today's digital landscape, data privacy is more critical than ever. As powerful language models like GPT-4 become increasingly...

Aug 4, 2024 🖱️ 8



See all from Bibek Poudel

Recommended from Medium



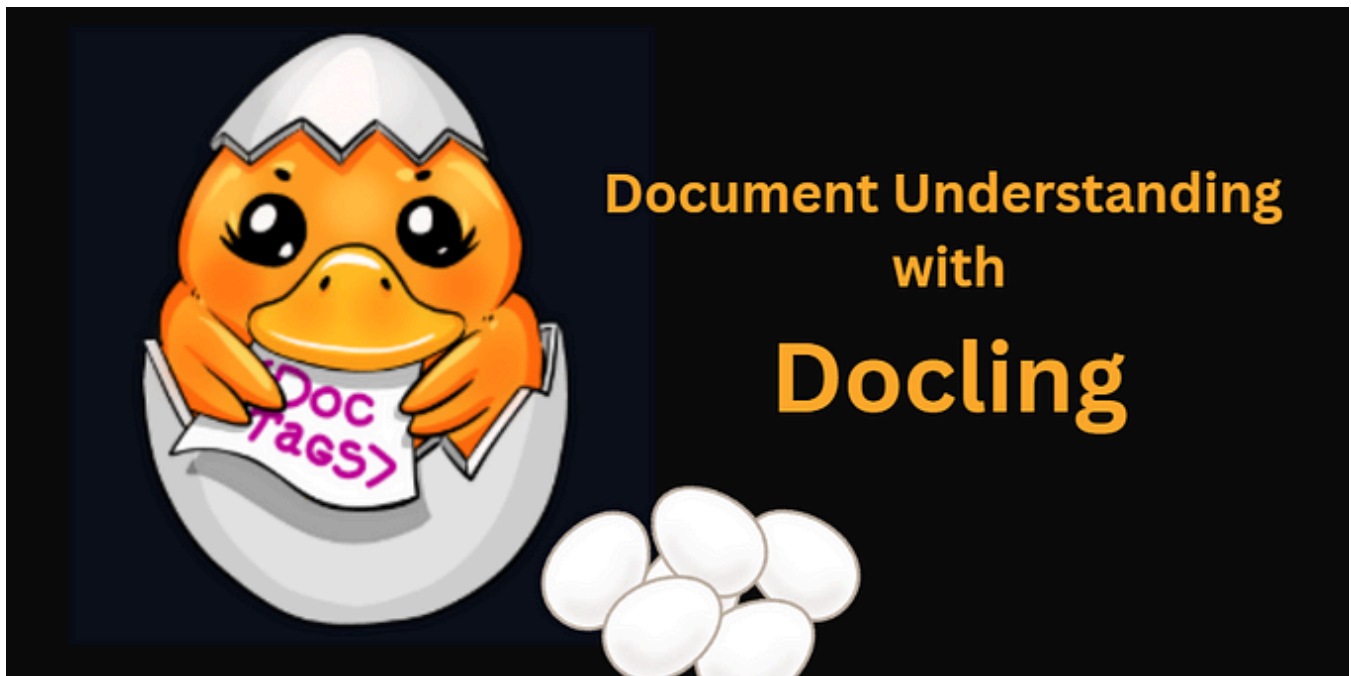
 In Data And Beyond by TONI RAMCHANDANI 

SmolDockling—Hugging Face’s Tiny OCR & Document Understanding Model

In a world obsessed with scaling LLMs to 70 billion parameters, Hugging Face did something wild—they went small. Really small. Enter...

★ Mar 24 🖱️ 93





 Harisudhan.S

SmolDocling : A Compact Vision Language Model

Simple documents are easy to manage. For instance, if we have a PDF, we can use a programming language to read it line by line and extract...

Mar 21  155  3



 In AI Mind by Vasukumar Palanisamy

How to extract the data from the Invoice Image in JSON form

Extracting the Invoice data from the Image by using the Qwen Vision Language model.

Oct 2, 2024 47

```
<ecel><nt><fcel>ktransformers-UD-IQ2_XXS<fcel>/<ecel><fcel>2.92<fcel>3.68 /<ecel><nt><fcel>ktransformers-Q5_K_S<fcel>/<ecel><fcel>0.57<fcel>1.42 /<ecel><nl><fcel>Ilama.cpp-UD-IQ2_XXS<ecel><ecel><ecel><fcel>340<ecel><nl><fcel>Ilama.cpp-Q5_K_S<ecel><ecel><ecel><fcel>320<ecel><nl><fcel>ktransformers-UD-IQ2_XXS<fcel>/<ecel><fcel>550<fcel>555 /<ecel><nl><fcel>ktransformers-Q5_K_S<fcel>/<ecel><fcel>450<fcel>458 /<ecel><nl><fcel>standby Watt<ecel><ecel><fcel>152<ecel><fcel>74<nl></otsl><end_of_utterance>
| Through | 2690 320GB | 2690+P40*2 | 2690+2080*2 | 7k62 512GB | 7k62+2080*2 | | |
|---|---|---|---|---|---|---|---|
| Ilama.cpp-UD-IQ2_XXS | 1.4 | | | | 4.88 | |
| Ilama.cpp-Q5_K_S | 0.66 | | | | 3.58 | |
| ktransformers-UD-IQ2_XXS | / | | | 2.92 | 3.68 / | |
| ktransformers-Q5_K_S | / | | | 0.57 | 1.42 / | |
| Ilama.cpp-UD-IQ2_XXS | | | | | 340 | |
| Ilama.cpp-Q5_K_S | | | | | 320 | |
| ktransformers-UD-IQ2_XXS | / | | | 550 | 555 / | |
| ktransformers-Q5_K_S | / | | | 450 | 458 / | |
| standby Watt | | | | 152 | | 74 | |
|-----|-----|-----|-----|-----|-----|
Generation on cuda took 8.99 s
(smoldocling) lwei@7k62:~$ python image-SmolDocling.py DeepSeek-R1.png --device cpu
Processing image file: DeepSeek-R1.png
Truncation was not explicitly activated but 'max_length' is provided a specific value, please use 'truncation=True' to explicitly truncate examples to max length. Defaulting to 'longest_first' truncation strategy. If you encode pairs of sequences (GLUE-style) with the tokenizer you can select this strategy more precisely by providing a specific strategy to 'truncation'.
Prepare inputs took 0.23 s
<doctag><otsl><loc_3><loc_0><loc_500><loc_500><ecel><ched>2690 320GB<ched>2690+P40*2<ched>2690+2080*2<ched>7k62 512GB<ched>7k62+2080*2<nl><rhed>Ilama.cpp-UD-IQ2_XXS<fcel>1.4<fcel><fcel>4.88<fcel>4.96<fcel>Ilama.cpp-Q5_K_S<nl><rhed>0.66<fcel><fcel>3.58<fcel>3.54<fcel>ktransformers-UD-IQ2_XXS<fcel>/<nl><rhed>2.92<fcel>3.68<fcel>/<fcel>5.67<fcel>ktransformers-Q5_K_S<fcel>/<nl><rhed>0.57<fcel>1.42<fcel>/<fcel>4.27<fcel>Ilama.cpp-UD-IQ2_XXS<fcel>340<nl><rhed>320<fcel>480<fcel>ktransformers-UD-IQ2_XXS<fcel>/<fcel>550<nl><rhed>450<fcel>555<fcel>/<fcel>570<fcel>ktransformers-Q5_K_S<fcel>/<nl><rhed>152<fcel>458<fcel>/<fcel>520<fcel>standby Watt<nl></otsl><end_of_utterance>
| | 2690 320GB | 2690+P40*2 | 2690+2080*2 | 7k62 512GB | 7k62+2080*2 |
|-----|-----|-----|-----|-----|-----|
| Ilama.cpp-UD-IQ2_XXS | 1.4 | <fcel> | 4.88 | 4.96 | Ilama.cpp-Q5_K_S |
```

Wei Lu

Convert PDF to text (markdown) with SmolDocling

SmolDocling is a vision-to-text model with only 256M parameters, so its inference resource requirements should be much lower than olmOCR...

Mar 25 27 1



Naeem Hadiq

Building an On premise RFP Response creation stack Utilising Docling, Ollama, Deepseek R1|...

In today's fast-paced business landscape, responding to Requests for Proposals (RFPs) efficiently can make or break an organization's...

Jan 28



In Data Science Collective by Buse Şenol

SmolDocling: A New Era in Document Processing — OCR

A model that outperforms its competitors 27 times its size with the DocTags format



Mar 24



713



2



See more recommendations